Bidirectional Sign Language Translation System

Complete Build Guide: Sign to Text/Speech and Text/Speech to an AI Signing Avatar

---

Python | MediaPipe | scikit-learn | Three.js

Prepared 11 October 2026

Contents

1. Project Overview
2. Key Decisions
3. Architecture and Folder Structure
4. Environment Setup
5. Training Data Without Recording Videos
6. Converting Images to Landmarks
7. Module A: Sign to Text/Speech
8. Module B: Text/Speech to Sign
9. AI Character (Avatar) That Performs Signs
10. Limitations and Common Pitfalls
11. Timeline and Deliverables Checklist
12. Next Steps and Sources

1. Project Overview

Goal. Build a low-cost, software-only system for real-time, two-way communication between a sign language user and a non-signer, using only a webcam, microphone and speaker. No gloves or special sensors are needed.

1.1 The two directions

Direction	What it does	Core technique
Sign to Text/Speech (Module A)	Reads hand gestures from the webcam, recognizes fingerspelled letters, assembles words, shows text and speaks it.	MediaPipe hand landmarks + a supervised classifier + word-assembly logic + text-to-speech.
Text/Speech to Sign (Module B)	Takes typed or spoken input and shows it as sign language so a signer can understand.	Speech-to-text, text normalization, letter-to-pose mapping, and an animated avatar (or images/clips).

1.2 Scope

In scope (MVP)	Out of scope (stretch goals)
Static fingerspelling alphabet recognition (one hand pose per letter) Word formation from sequential letters Text-to-speech and speech-to-text Text/speech-to-sign output via an avatar, images or clips A basic GUI for live demonstration	Continuous / sentence-level sign recognition (needs facial expression and body motion; research level) Multiple sign language dialects at once Mobile app deployment

1.3 Technology stack

Area	Tools
Language	Python 3.10 or 3.11 (MediaPipe can lag behind the newest Python)
Computer vision	OpenCV, MediaPipe Hands (21-point landmarks)
Machine learning	scikit-learn (Random Forest, SVM) or a small PyTorch/TensorFlow MLP
Speech	pyttsx3 (offline TTS) or gTTS; SpeechRecognition or OpenAI Whisper for STT
Avatar / GUI	Three.js in a web page served by Flask or FastAPI (or Streamlit with embedded HTML); Tkinter for the simple webcam panel
Data handling	NumPy, Pandas, joblib

2. Key Decisions

2.1 ASL or ISL?

Recommendation: start with ASL fingerspelling for the MVP. ASL fingerspelling is mostly one-handed and has plenty of public data. Indian Sign Language (ISL) fingerspelling uses mostly two hands, so the feature vector doubles from 63 to 126 values and classification and avatar posing both get harder. Build the full pipeline with ASL first, then swap the dataset and extend to two hands for ISL if time allows.

2.2 Dataset strategy

Do not use Sign Language MNIST: it is 28x28 grayscale and does not match live webcam input. Because you do not want to record videos, use an existing image dataset and run MediaPipe over it once, offline, to create a landmark CSV. Details are in Section 5.

2.3 GUI approach

Because the avatar needs smooth 3D rendering, use a small **web app** (Flask or FastAPI serving a Three.js page) for the main interface. Python handles speech, text normalization, mapping and recognition. Tkinter is fine for a quick webcam-only prototype, but it is poor for 3D.

3. Architecture and Folder Structure

```
SIGN -> TEXT/SPEECH
  webcam -> MediaPipe landmarks -> normalize -> classifier -> letter
         -> hold/gap word assembly -> text -> pyttsx3 speech

TEXT/SPEECH -> SIGN
  mic -> speech-to-text -> normalize (a-z, spaces) -> letters
      -> pose per letter -> smooth animation -> avatar hand/character
```

```
sign-translator/
|-- data/
|   |-- dataset/            # downloaded images, one folder per letter
|   `-- landmarks.csv       # generated from the images
|-- models/
|   `-- gesture_model.pkl
|-- src/
|   |-- images_to_landmarks.py
|   |-- train.py
|   |-- recognize.py        # live sign -> text
|   |-- assemble.py         # letters -> words
|   |-- tts_stt.py
|   |-- text_to_sign.py
|   `-- export_poses.py     # landmark CSV -> poses.json for the avatar
|-- web/
|   |-- index.html          # Three.js avatar page
|   `-- poses.json
|-- app.py                  # Flask / FastAPI server
`-- README.md
```

4. Environment Setup

```
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install opencv-python mediapipe numpy pandas scikit-learn
pip install pyttsx3 SpeechRecognition pyaudio joblib flask
```

Use Python 3.10 or 3.11. If pyaudio fails to install on Windows, install a prebuilt wheel or use **pipwin install pyaudio**.

5. Training Data Without Recording Videos

Recording is not required. Download an existing image dataset, take about 300 to 500 images per class, and convert them to landmark vectors once. You then train on the CSV exactly as if you had recorded the data yourself.

5.1 Datasets found

Dataset	Lang.	Size and format	Notes
ASL Alphabet (Kaggle; search the title, believed to be by grassknoted)	ASL	87,000 training images, 200x200 px, 29 folders	A-Z plus SPACE, DELETE, NOTHING (useful for word assembly). **Recommended for MVP.**
smart_gestures (PyPI package)	ASL	Ready-made hand landmarks CSV, 63 features per sample	Zero-effort option. Size and quality not verified; test before relying on it.
Sign Language MNIST (Kaggle)	ASL	34,627 images, 28x28 grayscale, 24 letters	J and Z missing; low resolution. Avoid for MediaPipe.
ISL Fingerspelling Image Dataset (IEEE DataPort)	ISL	About 14,000 images, 35 classes (A-Z, 1-9), ~400 per class	3 individuals, two backgrounds, varied lighting. May need a free account.
Static ISL gestures (Mendeley Data)	ISL	Over 1,00,000 images: alphabet, Hindi vowels, numerals	Different age groups and backgrounds. Read the usage terms first.
ISL landmark arrays (IEEE DataPort)	ISL	NumPy (x,y,z) for both hands, 30-frame sequences, all letters except R	Already in MediaPipe format; pick single frames per sequence.
ISL images with Google Drive link (GitHub)	ISL	35 classes, 1,200 images each	Built for an older bag-of-words pipeline; images only.

5.2 How to use them

1. Pick one dataset and sample about 300 to 500 images per class.
2. Convert images to landmarks once (Section 6) and train on the CSV.
3. Reuse one clean image per letter from the same dataset as the A-Z library for Module B.

5.3 Caveats

- **Detection failures:** MediaPipe misses the hand in some tight-cropped or blurry images. Skip them and check per-class counts.
- **Domain gap:** datasets have few people and plain backgrounds. Landmarks reduce this a lot, but test on your own hand early; live accuracy will be lower than held-out accuracy.
- **Leakage:** random splits put near-duplicate frames from one person in both train and test, which inflates accuracy. Split by person if IDs exist, otherwise state this limitation in the report.
- **Optional top-up:** if a few letters do badly live, record about 30 samples for just those letters and append them to the CSV.
- **Licensing:** check every dataset's terms and cite it in the report.

6. Converting Images to Landmarks

Expected folder layout: `dataset/A/img1.jpg`, `dataset/B/...`. The script detects the hand, normalizes the 21 points and writes a 63-feature CSV.

```
import cv2, csv, os, random
import mediapipe as mp

DATA_DIR, OUT, PER_CLASS = "dataset", "landmarks.csv", 400
hands = mp.solutions.hands.Hands(static_image_mode=True, max_num_hands=1,
                                 min_detection_confidence=0.5)
```

```
def normalize(lm):
    base = lm[0]                                    # wrist = origin
    pts = [(p.x - base.x, p.y - base.y, p.z - base.z) for p in lm]
    m = max(max(abs(v) for v in p) for p in pts) or 1   # scale invariance
    return [v / m for p in pts for v in p]

with open(OUT, "w", newline="") as f:
    w = csv.writer(f)
    w.writerow([f"f{i}" for i in range(63)] + ["label"])
    for label in sorted(os.listdir(DATA_DIR)):
        files = os.listdir(os.path.join(DATA_DIR, label))
        random.shuffle(files)
        kept = 0
        for name in files:
            if kept >= PER_CLASS: break
            img = cv2.imread(os.path.join(DATA_DIR, label, name))
            if img is None: continue
            res = hands.process(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
            if res.multi_hand_landmarks:
                w.writerow(normalize(res.multi_hand_landmarks[0].landmark) + [label])
                kept += 1
        print(label, kept)
```

Important: use the exact same **normalize()** function in the live recognizer. A different normalization will make the model fail on webcam input.

7. Module A: Sign to Text/Speech

7.1 Train and benchmark the classifier

- Split 80/20, stratified. Start with Random Forest, then compare SVM and a small MLP.
- Report accuracy, a confusion matrix and per-letter F1 in a comparison table (the brief asks for a documented benchmark).
- Expect confusion between similar signs such as M/N/T and U/V/R; mention these in the report.
- Save the chosen model with joblib.

```
import pandas as pd, joblib
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

df = pd.read_csv("landmarks.csv")
X, y = df.drop(columns="label"), df["label"]
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)

models = {"RandomForest": RandomForestClassifier(n_estimators=300, n_jobs=-1),
          "SVM": SVC(kernel="rbf", C=10, probability=True)}
for name, m in models.items():
    m.fit(Xtr, ytr)
    print(name); print(classification_report(yte, m.predict(Xte)))
joblib.dump(models["RandomForest"], "models/gesture_model.pkl")
```

7.2 Live recognition

Per frame: detect landmarks, normalize, call `predict_proba`, and accept the top letter only if confidence is above about 0.8. Flip the frame for display, but be consistent with how the training images were oriented.

7.3 Word and sentence assembly

- **Hold logic:** accept a letter only after it is predicted consistently for about 15 frames (about 0.5 s).

- **Debounce:** do not add the same letter again while the pose is held; require the hand to change or leave first.
- **Gap logic:** if no hand is seen for about 1.5 s, commit the word and insert a space.
- **Corrections:** provide a delete gesture (or GUI button) and a clear button.

```
HOLD_FRAMES, GAP_SECONDS = 15, 1.5
# per frame:
#   if hand: if letter == candidate: count += 1 else: candidate, count = letter, 1
#            if count == HOLD_FRAMES and (letter != last_committed or hand_left_since_commit):
#                commit(letter)
#   if no hand for GAP_SECONDS: commit_space_and_word()
```

7.4 Text-to-speech

Use `pyttsx3` (offline). Run it in a separate thread so the video loop does not freeze while speaking.

8. Module B: Text/Speech to Sign

Step	Technique	Tool
Speech-to-text	Transcribe microphone audio	SpeechRecognition (recognize_google) to start; Whisper for offline or better accuracy
Normalization	Lowercase, strip punctuation, keep a-z and spaces	Python string operations / NLTK
Sign mapping	Map each letter (and optionally whole common words) to a pose	poses.json (from landmarks) or image/clip library
Playback	Show the sequence in sync with the text, about 0.7 s per letter and a longer pause between words	Three.js avatar, or OpenCV/Tkinter for simple image playback

Whole-word shortcut. Add a small dictionary of 10 to 20 common words (hello, thanks, yes). Try the whole word first and fall back to fingerspelling. The simplest asset set is one clean image per letter taken from your dataset.

9. AI Character (Avatar) That Performs Signs

You already have 21-point landmarks for each letter. An avatar signing a letter is the same data run in reverse: take the landmarks for the letter and pose a 3D hand to match.

```
text/speech -> letters -> pose per letter -> animate between poses -> avatar hand moves
```

9.1 Three levels of ambition

Level	What you build	Effort	Look
1	3D skeleton hand (spheres and sticks) drawn from landmarks	1-2 days	Technical but works; best first milestone
2	Rigged 3D character (GLB or VRM) with finger bones rotated to match	3-5 days	A real AI character
3	Full-body signing with arms, face and grammar	Weeks or more	Research level; skip for this project

Build Level 1 first. The pose data and timing logic carry over unchanged when you upgrade to Level 2.

9.2 Step 1: one pose per letter (Python)

Take the **median** vector per letter from the landmark CSV. This is the canonical pose and removes noisy samples.

```
import pandas as pd, json

df = pd.read_csv("landmarks.csv")
poses = {}
for label, g in df.groupby("label"):
    med = g.drop(columns="label").median().values.reshape(21, 3)
    poses[label.lower()] = med.round(4).tolist()

json.dump(poses, open("web/poses.json", "w"))
```

9.3 Step 2: render and animate (Three.js)

Serve a small web page with Flask or FastAPI. Python does the speech, normalization and mapping; the browser does the 3D. The key to natural motion is easing the current pose toward the target pose every frame, instead of jumping.

```
// poses = { a: [[x,y,z] x21], b: ..., ... }
function playWord(word, msPerLetter = 700) {
  const queue = [...word.toLowerCase()].filter(c => poses[c]);
  let i = 0;
  setInterval(() => { if (i < queue.length) target = poses[queue[i++]]; }, msPerLetter);
}

function animate() {
  for (let j = 0; j < 21; j++)
    for (let k = 0; k < 3; k++)
      current[j][k] += (target[j][k] - current[j][k]) * 0.15;   // smoothing
  updateJointMeshes(current);          // move spheres, stretch sticks between joints
  renderer.render(scene, camera);
  requestAnimationFrame(animate);
}
```

Add a short neutral pose between repeated letters so double letters (the LL in HELLO) read as two separate letters.

9.4 Step 3: upgrade to a rigged character (Level 2)

1. **Get an avatar:** a free VRM or GLB model with a humanoid skeleton and finger bones (for example from Ready Player Me or VRoid Studio). Check each license.
2. **Load it** with GLTFLoader (plus @pixiv/three-vrm for VRM).
3. **Retarget landmarks to bones:** rotate each finger bone so it points the way the landmark chain does (code below).
4. **Frame the camera** on the upper body and hand, palm facing the viewer, so the sign is readable.

```
// for the bone between landmark a -> b, in the parent bone's local space
const dir = new THREE.Vector3(...pb).sub(new THREE.Vector3(...pa)).normalize();
bone.quaternion.setFromUnitVectors(restDirection, dir);
```

- Store each bone's **rest direction** once, from the avatar's T-pose.
- Apply rotations from the wrist outward, converting each landmark direction into the parent's local frame.
- Use `quaternion.slerp` toward the target for smooth motion.

9.5 What the avatar can and cannot do

- **Fingerspelling works well** because letters are static hand poses. This matches the MVP scope.

- **Dynamic signs** (J and Z in ASL) need motion. Animate them as short pose sequences or leave them out.
- **ISL** poses both hands for many letters, which doubles the posing work; another reason to start with ASL.
- **Whole words and sentences** involve arm movement, face and grammar. For a small vocabulary, extract landmark sequences from a few clips and replay them, and state this limitation clearly in the report.
- **Existing toolkits:** HamNoSys/SiGML-based avatar tools exist (CWASA is one), but their libraries mainly target languages like British Sign Language, and ISL coverage is uncertain. Treat as a research lead only.

10. Limitations and Common Pitfalls

Issue	What to do
J and Z need motion in ASL	Exclude them, add a simple motion rule, or list as a limitation.
Left vs right hand	Mirror landmarks for left-handed users, or train on both.
Overfitting to one person or session	Test on a different person; report live accuracy as well as held-out accuracy.
Webcam mirror mismatch	Flip the frame for display, but feed MediaPipe the same orientation as the training images.
Similar letters (M/N/T, U/V/R)	Show the confusion matrix; consider confidence thresholds and the hold logic to reduce errors.
Lighting and background	Landmarks help, but test in your demo environment.
Camera and TTS freezing the UI	Run camera, TTS and STT in separate threads.
Static signs only	State clearly that sentence-level sign language is out of scope.

11. Timeline and Deliverables Checklist

11.1 Suggested six-week plan

Week	Tasks
1	Literature review, choose ASL or ISL, download dataset, set up environment, run images-to-landmarks conversion.
2	Train and benchmark classifiers (Random Forest, SVM, MLP); confusion matrix; save the model.
3	Live recognition, hold/gap/debounce word assembly, text-to-speech.
4	Speech-to-text, normalization, poses.json export, Level 1 avatar in Three.js with smooth animation.
5	Connect Python and the avatar page, integrate both modes in one app, test with other people; optional Level 2 rigged character.
6	README, demo video, final report with benchmark table and limitations.

11.2 Deliverables checklist

- Working Python application demonstrating both directions of translation.

- Trained gesture classifier with a documented accuracy benchmark (table, confusion matrix, per-letter F1).
- Demo video or report: live sign to text to speech, and typed or spoken text to avatar signing.
- Source code with a clear README covering setup, dependencies and usage.
- Report sections for dataset sources and licenses, limitations, and future work.

12. Next Steps and Sources

12.1 Immediate next steps

1. Decide ASL or ISL (ASL recommended for the MVP).
2. Download the dataset and run the images-to-landmarks script.
3. Train the classifier and record its benchmark.
4. Build the live recognizer and word assembly.
5. Export poses.json and build the Level 1 avatar.

12.2 Sources

- ASL alphabet recognizer repository: github.com/parakh-gupta/Sign_language_alphabet_recognizer
- ASL Alphabet dataset (HyperAI mirror of the Kaggle dataset): hyper.ai/de/datasets/32503
- ASL-MNIST dataset card: docs.voxel51.com/dataset_zoo/datasets_hf/american_sign_language_mnist.html
- smart_gestures on PyPI: pypi.org/project/smart_gestures
- ISL Fingerspelling Image Dataset, IEEE DataPort: ieee-dataport.org/documents/isl-fingerspelling-image-dataset
- Indian Sign Language datasets, IEEE DataPort: ieee-dataport.org/keywords/indian-sign-language
- Static ISL gestures (English alphabet, Hindi vowels, numerals), Mendeley Data: data.mendeley.com/datasets/7tsw22y96w
- Indian-sign-language-recognition repository: github.com/Karthikeyu/Indian-sign-language-recognition